

Experiment-03

Aim:

- A. You are given with the Employee schema; the task is to find the highest score for each respective department.
For eg: department D1 have the highest score as 5.28. Hence in the output, final updated score for D1 should be 5.28. Same with the other departments i.e., D2, D3 and D4.
- B. A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.
- C. A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the top three highest distinct salary values within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Code and Output:

A.

The screenshot displays a SQL IDE interface. At the top, a toolbar contains icons for file operations, execution, and help. Below the toolbar, the 'Query' tab is active, showing a SQL script. The script defines an 'Employee' table with columns 'EID', 'DEPT', and 'SCORES', and then inserts eight rows of data. The 'Query History' tab is also visible. At the bottom, the 'Data Output' tab shows the execution result: 'INSERT 0 8', indicating that 8 rows were inserted. A message below states 'Query returned successfully in 48 msec.'

```
1 CREATE TABLE Employee
2 (
3     EID INT PRIMARY KEY,
4     DEPT VARCHAR(10),
5     SCORES DECIMAL(5,2)
6 );
7
8 INSERT INTO Employee (EID, DEPT, SCORES)
9 VALUES
10 (1, 'D1', 1.00),
11 (2, 'D1', 5.28),
12 (3, 'D1', 4.00),
13 (4, 'D2', 8.00),
14 (5, 'D1', 2.50),
15 (6, 'D2', 7.00),
16 (7, 'D3', 9.00),
17 (8, 'D4', 10.20);
```

Data Output Messages Notifications

INSERT 0 8

Query returned successfully in 48 msec.

The screenshot shows a database query editor with a dark theme. The top toolbar contains icons for file operations, query execution, and settings. The query text is as follows:

```
1 UPDATE Employee AS E
2 SET SCORES =
3 (
4 SELECT MAX(SCORES)
5 FROM Employee
6 WHERE DEPT = E.DEPT
7 );
8
9 SELECT * FROM Employee;
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 8 rows. The table has four columns: 'eid' (integer, primary key), 'dept' (character varying), 'scores' (numeric), and an unnamed column. The data is as follows:

	eid [PK] integer	dept character varying (10)	scores numeric (5,2)	
1	1	D1	5.28	
2	2	D1	5.28	
3	3	D1	5.28	
4	4	D2	8.00	
5	5	D1	5.28	
6	6	D2	8.00	
7	7	D3	9.00	
8	8	D4	10.20	

B.

The image shows two screenshots of a database query editor interface. The top screenshot displays the execution of three CREATE TABLE statements. The bottom screenshot displays the execution of three INSERT INTO statements.

Top Screenshot: CREATE TABLE Statements

Query:

```
1 CREATE TABLE SalesPerson (  
2     seller_id INT PRIMARY KEY,  
3     name VARCHAR(50),  
4     city VARCHAR(50),  
5     commission INT  
6 );  
7  
8 CREATE TABLE Company (  
9     com_id INT PRIMARY KEY,  
10    name VARCHAR(50),  
11    city VARCHAR(50)  
12 );  
13  
14 CREATE TABLE Orders (  
15     order_id INT PRIMARY KEY,  
16     order_date DATE,  
17     com_id INT,  
18     seller_id INT,  
19     amount INT,  
20     FOREIGN KEY (com_id)  
21         REFERENCES Company(com_id),  
22     FOREIGN KEY (seller_id)  
23         REFERENCES SalesPerson(seller_id)  
24 );
```

Data Output:

```
CREATE TABLE  
  
Query returned successfully in 121 msec.
```

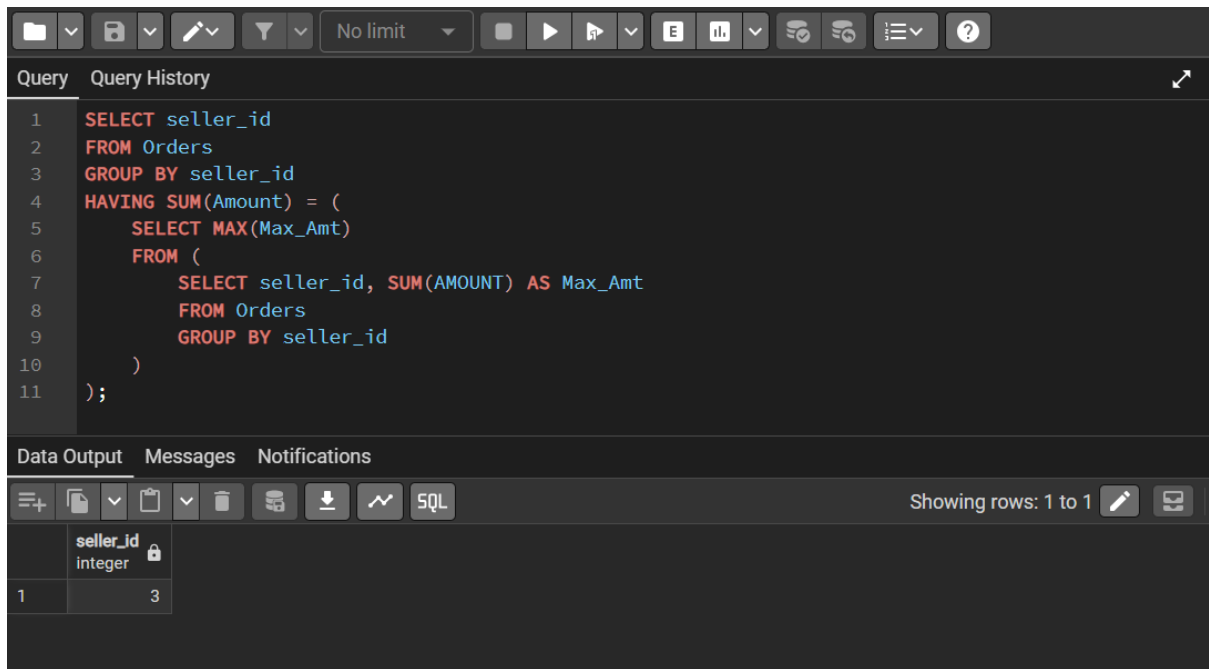
Bottom Screenshot: INSERT INTO Statements

Query:

```
1 INSERT INTO SalesPerson  
2 VALUES  
3 (1,'John','New York',15),  
4 (2,'Amy','Los Angeles',13),  
5 (3,'Mark','Chicago',12),  
6 (4,'Pam','Boston',15);  
7  
8 INSERT INTO Company  
9 VALUES  
10 (1,'RED','Boston'),  
11 (2,'ORANGE','New York'),  
12 (3,'YELLOW','Boston'),  
13 (4,'GREEN','Austin');  
14  
15 INSERT INTO Orders  
16 VALUES  
17 (1,'2024-01-10',1,1,1200),  
18 (2,'2024-01-12',2,1,800),  
19 (3,'2024-01-15',3,2,2500),  
20 (4,'2024-01-18',1,3,1500),  
21 (5,'2024-01-22',4,2,700),  
22 (6,'2024-01-25',2,3,2000),  
23 (7,'2024-01-28',3,4,3000),  
24 (8,'2024-01-30',4,4,200);
```

Data Output:

```
INSERT 0 8  
  
Query returned successfully in 64 msec.
```



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL query:

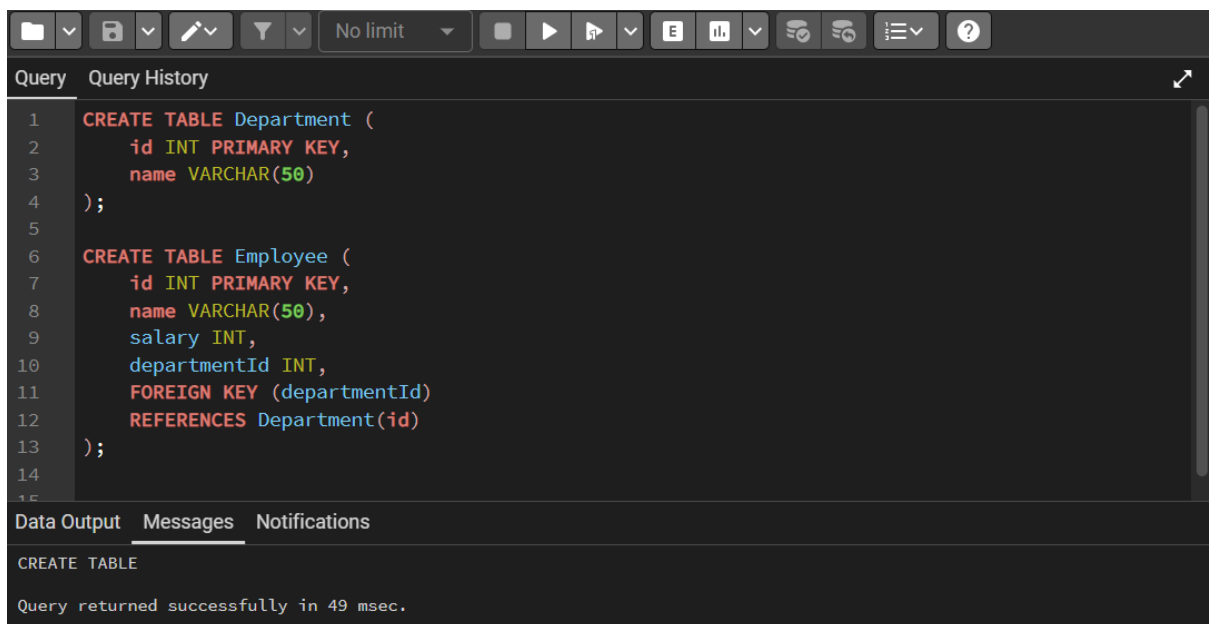
```
1 SELECT seller_id
2 FROM Orders
3 GROUP BY seller_id
4 HAVING SUM(Amount) = (
5     SELECT MAX(Max_Amt)
6     FROM (
7         SELECT seller_id, SUM(AMOUNT) AS Max_Amt
8         FROM Orders
9         GROUP BY seller_id
10    )
11 );
```

The 'Data Output' tab is also active, showing the results of the query. The output is a table with one column, 'seller_id', and one row with the value 3.

seller_id
3

The status bar at the bottom indicates 'Showing rows: 1 to 1'.

C.

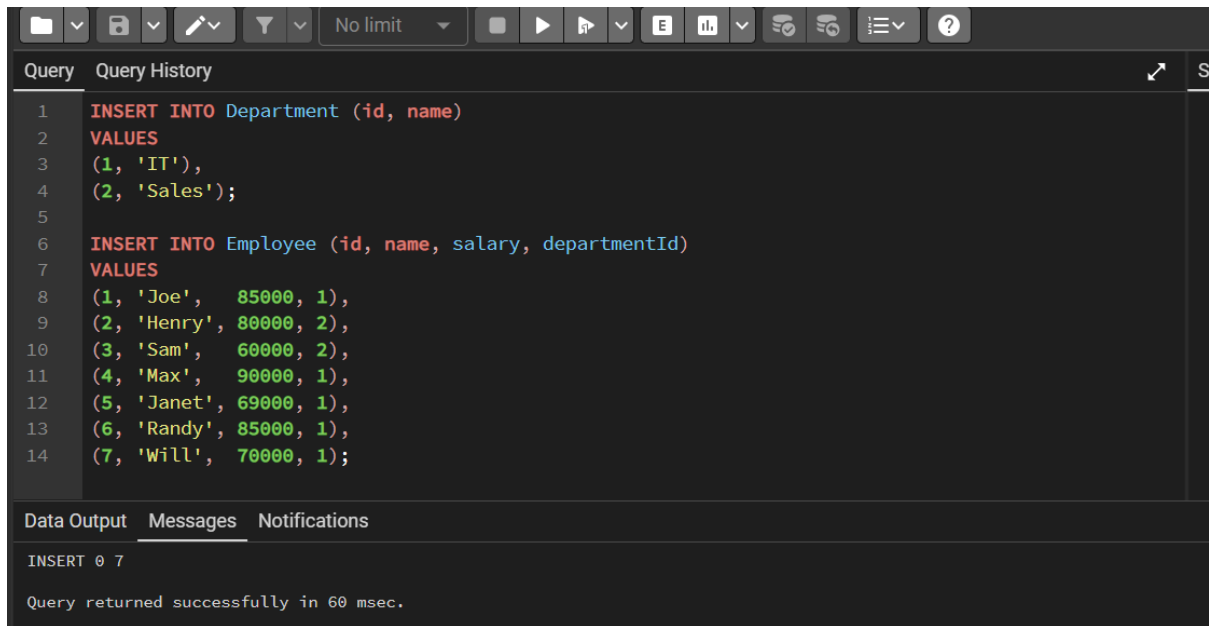


The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL query:

```
1 CREATE TABLE Department (
2     id INT PRIMARY KEY,
3     name VARCHAR(50)
4 );
5
6 CREATE TABLE Employee (
7     id INT PRIMARY KEY,
8     name VARCHAR(50),
9     salary INT,
10    departmentId INT,
11    FOREIGN KEY (departmentId)
12    REFERENCES Department(id)
13 );
```

The 'Data Output' tab is also active, showing the results of the query. The output is a message indicating that the query was successful.

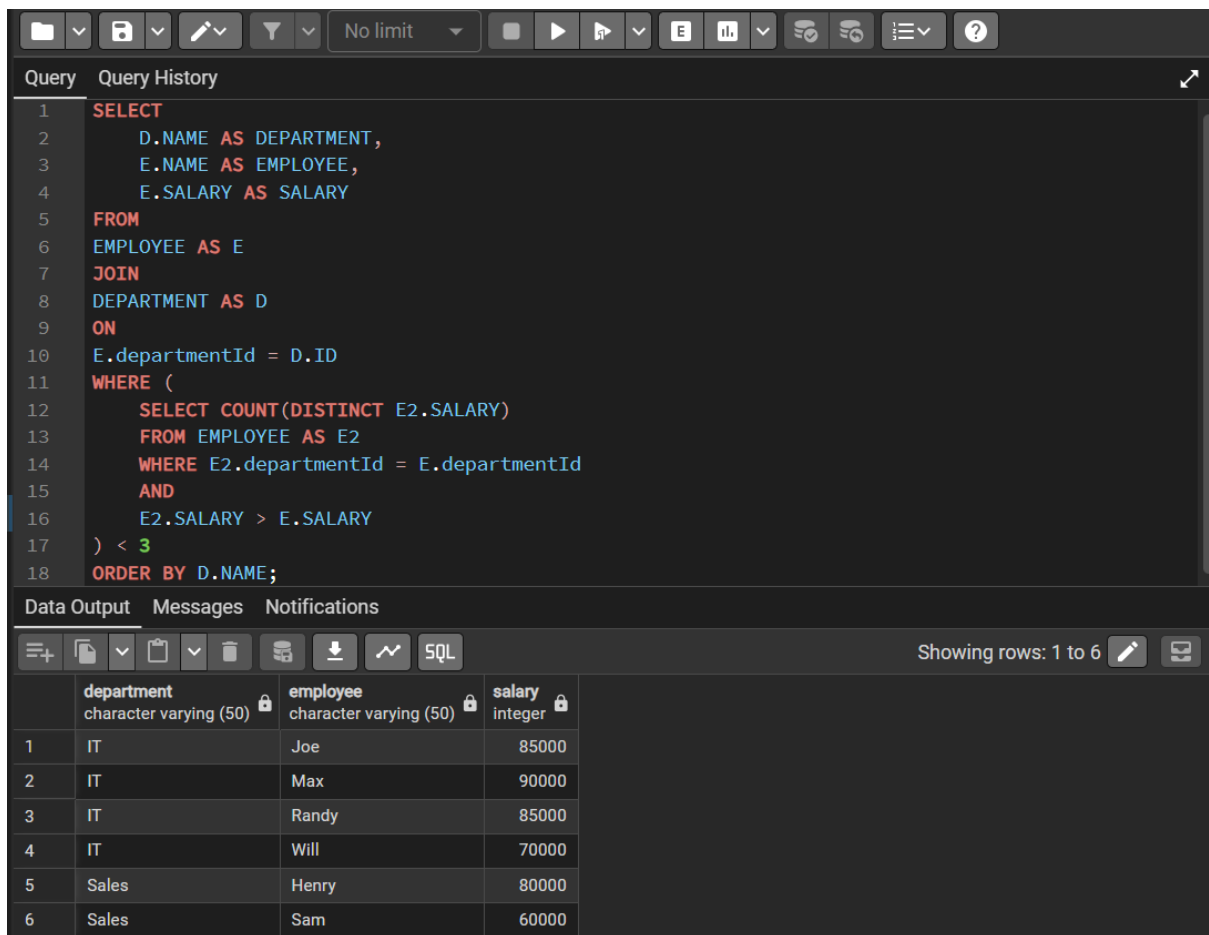
```
CREATE TABLE
Query returned successfully in 49 msec.
```



The screenshot shows a SQL IDE interface with a dark theme. The top toolbar includes icons for file operations, query execution, and settings. The main query editor contains the following SQL code:

```
1 INSERT INTO Department (id, name)
2 VALUES
3 (1, 'IT'),
4 (2, 'Sales');
5
6 INSERT INTO Employee (id, name, salary, departmentId)
7 VALUES
8 (1, 'Joe', 85000, 1),
9 (2, 'Henry', 80000, 2),
10 (3, 'Sam', 60000, 2),
11 (4, 'Max', 90000, 1),
12 (5, 'Janet', 69000, 1),
13 (6, 'Randy', 85000, 1),
14 (7, 'Will', 70000, 1);
```

Below the query editor, the 'Data Output' tab is active, displaying the message: 'INSERT 0 7' and 'Query returned successfully in 60 msec.'



The screenshot shows the same SQL IDE interface. The main query editor contains the following SQL code:

```
1 SELECT
2     D.NAME AS DEPARTMENT,
3     E.NAME AS EMPLOYEE,
4     E.SALARY AS SALARY
5 FROM
6     EMPLOYEE AS E
7 JOIN
8     DEPARTMENT AS D
9 ON
10    E.departmentId = D.ID
11 WHERE (
12     SELECT COUNT(DISTINCT E2.SALARY)
13     FROM EMPLOYEE AS E2
14     WHERE E2.departmentId = E.departmentId
15     AND
16     E2.SALARY > E.SALARY
17 ) < 3
18 ORDER BY D.NAME;
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 6 rows. The table has 4 columns: department, employee, salary, and an unnamed column. The data is as follows:

	department character varying (50)	employee character varying (50)	salary integer	
1	IT	Joe	85000	
2	IT	Max	90000	
3	IT	Randy	85000	
4	IT	Will	70000	
5	Sales	Henry	80000	
6	Sales	Sam	60000	

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with subqueries in PostgreSQL.
- Practiced commands like GROUP BY and ORDER BY in PostgreSQL